NAME

journal - process log for writing a C compiler

DESCRIPTION

Documenting the process of writing my C compiler.

ENTRIES

July 26, 2026

Converted code to KNF and added a new function to do that. Converted the journal into mandoc format like I was back in AT&T Bell Labs.

View the file using mandoc:

```
mandoc journal.1 | less
```

Convert to PDF:

```
mandoc -T pdf journal.1 > journal.pdf
```

Check for syntax errors:

```
mandoc -T lint journal.1
```

July 22, 2026

Got the test suite working - arguments passed into the test suite were getting passed into the driver and lexer. Moving on to the parser now.

July 19, 2026

Struggling with getting the test suite to work with my driver. Reading **sed** & **awk** to distract me from fixing the test suite.

July 4, 2026

Using iPad Pro as a "dumb terminal" with ShellFish client and OpenBSD server works really well.

Special characters on iPad CH Magic Keyboard:

Option 5
[

Option 6

]

Option 7
|

Option 8
{

Option 9
}

Option n
~

Option SHIFT 7
\

July 3, 2026
Vi notes.

Find and replace in current line:

:s/CATS/DOGS/g

Find and replace in entire file:

:%s/CATS/DOGS/g

Visual Block Mode procedure:

1. Press **Ctrl+v**.
2. Use **j** and **k** keys to select rows.
3. Press **Shift+i** to enter insert mode.
4. Type the character.
5. Press **Esc** twice.

July 2, 2026
Applying kernel source file style to my scripts.

Refer to style(9).

June 28, 2026
Confused about different types of pattern matching that can be done. Globbing and extended globbing are designed for file name matching, so I am going to use extended regex for the lexer.

Standard Globbing

*****
Any string of characters. ‘file*.txt’ matches *file.txt*, *file1.txt*.

?
Exactly one character. ‘image?.png’ matches *image1.png*, *imageA.png*.

[...]
Any one character, including ranges. ‘log_[0-9].txt’ matches *log_0.txt*, *log_1.txt*.

[!...]
or **[^...]** Matches any one character not enclosed in brackets. ‘data_[!0-9].csv’ matches *data_a.csv*.

[:space:], **[:digit:]**, **[:lower:]**, **[:upper:]**, **[:alpha:]**, **[:alnum:]**
POSIX character classes. ‘file_[[:digit:]].txt’ matches *file_7.txt*.

Extended Globbing

Uses a quantifier followed by a list of patterns, separated by OR ‘|’.

?(pattern-list)
Zero or ONE occurrence of patterns. ‘file?(test).txt’ matches *file.txt* and *filetest.txt*.

***(pattern-list)**
Zero or MORE occurrences of patterns. ‘data*(0).csv’ matches *data0.csv* and *data000.csv*.

+(pattern-list)
One or more occurrences. ‘track+(0).mp3’ matches *track0.mp3* but won’t match *track.mp3*.

@(pattern-list)

Exactly one of the given patterns. ‘report.@(pdf|txt).doc’ matches *report.pdf*, *report.txt* but won’t match *report.jpg*.

!(pattern-list)
Anything except the given patterns (can be combined).

Extended Globbing Examples:

- ‘’*([0-9])?(.csv)’’ finds strings that optionally end with .csv (e.g., 123 and 564.csv).

- ‘rm ’!(*.jpg|*.png)’’ deletes all files except jpg and png images.

June 20, 2026
Lexer requires proper understanding of regular expressions, so I started reading "Mastering Regular Expressions". Two books to read before picking up the compiler book again.

Got a full list of English words and full works of Shakespeare to practice RegEx.

June 14, 2026
Got the book from eBay, reading Chapter 1. Wrote the compiler driver. Wrote the lexer using Gemini, decided not to use LLMs, deleted the code, and am starting it from scratch.

May 23, 2026
Installed **ksh93** on my OpenBSD computer for writing the compiler. OpenBSD comes with **pdksh** (Public Domain Korn Shell), which lacks features that I need.

May 19, 2026
Got the Korn shell book from eBay, reading that.